

Executors, Parallelism & Task Queues

1. What's an executor?

The **Executor** is the core *mechanism* that the Scheduler uses to actually run tasks that are ready to be executed. It is the bridge between the Scheduler (which decides *what* to run and *when*) and the compute resources that perform the work.

- **Location:** The executor runs *inside* the Scheduler process.
 - **Responsibility:** When the Scheduler determines a task is ready (dependencies are met, state is queued), it passes the task command to the Executor. The Executor's job is to get that task executed.
 - **Key Concept:** The choice of Executor is the primary factor that determines Airflow's scalability and ability to run tasks in parallel.
-

2. Default Config

The executor is configured in the `airflow.cfg` file with the `executor` parameter. The default value in a standard pip installation is:

```
ini
```

```
executor = SequentialExecutor
```

This is a very important setting that defines the entire behavior of your Airflow deployment.

3. The Sequential Executor

- **How it works:** This executor runs **one single task at a time**. It handles task execution in a single process. It uses the Metadata Database to store task state and is the only executor that works with SQLite.
- **Use Case: Development and Testing only.** It is useless for any real workload because it offers zero parallelism.
- **Major Limitation:** Since it uses SQLite and SQLite does not support multiple writes, this executor cannot parallelize tasks even if you set parallelism higher. The database becomes a bottleneck.
- **Why it exists:** It's the simplest possible executor, requires no additional setup, and is the default to ensure a beginner can install Airflow and run a simple DAG without any extra components.

4. The Local Executor

- **How it works:** This executor runs tasks as **sub-processes on the same machine** as the Scheduler. It uses the parallelism config setting in airflow.cfg to determine the maximum number of task instances that can run simultaneously.
- **Use Case: Development, Testing, and Small Production workloads.** It is a significant step up from the SequentialExecutor as it allows for true parallelism on a single machine.
- **Requirement:** Requires a database that supports concurrent connections (like PostgreSQL or MySQL). It will **not** work with SQLite.
- **Limitation:** It is limited by the CPU and memory of the single machine it runs on. Scaling vertically (bigger machine) has a hard limit.

5. The Celery Executor

- **How it works:** This is a **distributed, asynchronous** executor. It uses a **message queue** (like Redis or RabbitMQ) to coordinate between the Scheduler and a pool of **Worker nodes** that can be spread across multiple machines.
 1. The Scheduler adds commands for tasks that are ready to the message queue.
 2. **Celery Worker** processes (on one or many machines) constantly listen to this queue.
 3. A worker picks up a task from the queue, executes it, and then reports the result back to the metadata database.
- **Use Case: Medium to Large Production workloads.** It allows you to scale horizontally by adding more worker machines to handle increased load.
- **Components Required:**
 - A supported database (PostgreSQL/MySQL).
 - A message broker (Redis or RabbitMQ).
 - One or more machines running the airflow celery worker command.

6. The Current Config

Let's check our current setup. Since we are using the official Docker Compose setup, we can look at its configuration.

The docker-compose.yaml file typically sets the executor via an environment variable. The standard setup often uses CeleryExecutor to demonstrate a production-like environment, even locally.

You can check this by running:

```
# Inside your Airflow Docker environment
airflow config get-value core executor
```

This command will likely return CeleryExecutor.

The compose file also defines services for:

redis: The message broker for Celery.

airflow-worker: A Celery worker node.

flower: A tool for monitoring Celery (more on this later).

7. Add the DAG `parallel_dag.py` into the `dags` folder

To demonstrate parallelism, we need a DAG with tasks that can run at the same time. Let's create a DAG that uses a `@task` decorator to create many parallel tasks.

DAG File: `dags/parallel_dag.py`

```
from airflow import DAG
from airflow.decorators import task
from datetime import datetime
```

```
default_args = {
    'start_date': datetime(2023, 1, 1),
}
```

```
with DAG(
    'parallel_dag',
    default_args=default_args,
    schedule_interval='@daily',
    catchup=False,
    max_active_runs=1,
) as dag:
```

```
    @task
```

```
    def start():
        return "Start"
```

```
    @task
```

```
def parallel_task(number: int):
    """A simple task that prints its number and simulates work."""
    print(f"Starting task number {number}")
    # Simulate work by sleeping for a few seconds
    import time
    time.sleep(5)
    print(f"Finished task number {number}")
    return f"Processed {number}"

@task
def end():
    return "End"

# This is the key: dynamically creating 10 parallel tasks
parallel_tasks = parallel_task.expand(number=list(range(1, 11)))

# Set up the dependencies
start() >> parallel_tasks >> end()
```

Key Line: `parallel_task.expand(number=list(range(1, 11)))`

This uses the `.expand()` method to create **10 separate task instances** (for numbers 1 through 10) that will all be eligible to run in parallel after the start task finishes.

8. Monitor your tasks with Flower

Flower is a real-time web-based monitoring tool for Celery. It is essential for managing and debugging a Celery-based Airflow deployment.

- **What it shows:**
 - **Workers:** List of all active workers, their status, and resource usage (CPU, memory).
 - **Tasks:** A list of all tasks, showing if they are queued, running, succeeded, or failed.
 - **Graphs:** Real-time graphs of task throughput, execution times, and worker activity.
 - **Detailed Views:** You can inspect individual tasks, see their arguments, and even revoke (cancel) running tasks.
- **Access:** If running with Docker Compose, Flower is usually available on port 5555 (e.g., <http://localhost:5555>).

9. Remove DAG examples

The Airflow Docker image comes with many example DAGs. For a clean production-like environment and to avoid clutter and confusion during testing, it's best to disable them.

This is typically done by setting the environment variable `AIRFLOW__CORE__LOAD_EXAMPLES` to 'false' in the `docker-compose.yaml` file for the Scheduler, Webserver, and Worker services. If you already did this during initial setup, the examples should be gone. If not, you can set it and restart the containers.

10. Running tasks on Celery Workers

With the `parallel_dag` running and the `CeleryExecutor` configured, the process is automatic:

1. The Scheduler parses the DAG and sees 10 `parallel_task` instances are ready.
 2. It places 10 task messages into the Redis queue.
 3. The `airflow-worker` container (our Celery worker) is continuously listening to this queue.
 4. The worker pulls tasks from the queue and executes them. By default, a worker will run multiple tasks in parallel (using sub-processes) based on its concurrency settings.
 5. You can watch this happen in real-time in the **Flower dashboard**, which shows tasks being consumed from the queue and moving to a "running" state.
-

11. What is a queue?

In the context of Celery, a **queue** is a named channel within the message broker. By default, all tasks are sent to a queue named `default`.

- **Why use multiple queues?** To prioritize work and assign specific types of tasks to specific groups of workers.
 - **Use Cases:**
 - **Priority:** A `high_priority` queue for critical reports and a `low_priority` queue for less important data mining tasks.
 - **Resource Isolation:** A `machine_learning` queue whose workers have GPUs installed, and a general queue for standard Python and SQL tasks.
 - **Team/Department Isolation:** Different teams can have their own dedicated worker pools listening on their own queues.
-

12. Add a new Celery Worker

To use multiple queues, we need workers listening on those queues. We can add a new worker to our Docker Compose setup.

Step 1: Edit docker-compose.yaml

Duplicate the airflow-worker service block, give it a new name, and change the command to listen to a specific queue.

```
# Original worker (listens to the default queue)

airflow-worker:

  &airflow-common

  # ... other settings ...

  command: celery worker


# NEW worker service for high priority tasks

airflow-worker-high-prio:

  <<: *airflow-common # Inherits all common settings

  hostname: airflow-worker-high-prio

  command: celery worker --queues high_priority # This is the crucial change

  # Optional: You can give it a different name in the UI

  environment:

    - AIRFLOW__CELERY__WORKER_HOSTNAME=airflow-worker-high-priority
```

Step 2: Deploy the new worker

Run `docker-compose up -d` to start the new `airflow-worker-high-prio` container. You will now see two separate workers in the Flower dashboard.

13. Create a queue to better distribute tasks

The queue itself is created simply by using its name. There is no explicit "create queue" command. The first time a task is sent to a queue (e.g., `high_priority`) or a worker starts listening to it, the queue is automatically created in the message broker.

14. Send a task to a specific queue

You assign a task to a queue using the `queue` parameter in the Operator or Task. Let's modify our DAG to send some tasks to the high priority queue.

Modified DAG: `dags/parallel_dag.py`

```
@task(queue='high_priority') # Assign this task to the high_priority queue

def critical_task(number: int):

    """This important task will be sent to the high_priority queue."""
```

```

print(f"Starting CRITICAL task number {number}")

import time

time.sleep(3)

print(f"Finished CRITICAL task number {number}")

return f"Critical {number} processed"

@task # This task uses the default queue
def normal_task(number: int):

    """This normal task will be sent to the default queue."""

    print(f"Starting normal task number {number}")

    import time

    time.sleep(5)

    print(f"Finished normal task number {number}")

    return f"Normal {number} processed"

# Create tasks on different queues

critical_tasks = critical_task.expand(number=list(range(1, 4))) # 3 critical tasks

normal_tasks = normal_task.expand(number=list(range(1, 6))) # 5 normal tasks

start() >> [critical_tasks, normal_tasks] >> end()

```

Now, when you run this DAG:

- 3 critical_task instances will be sent to the high_priority queue and will be picked up *only* by the airflow-worker-high-prio worker.
- 5 normal_task instances will be sent to the default queue and will be picked up by the original airflow-worker container.

This demonstrates how you can effectively isolate and manage different workloads within the same Airflow environment.